

Prudential SG Policy, Claims & Agent Book

Analytics Warehouse & Data Integration — Client Report

This report explains, in business terms, how the cleaned policy book is turned into a governed analytics warehouse: a structured data model built for reporting, and an integration pipeline that populates it reliably, repeatably, and with every number traceable back to source.

Prepared for	Client stakeholders / Analytics leadership
Source data	prudential_sg_cleaned_imputed.csv (47,500 policies, fully cleansed)
Deliverables covered	Warehouse Schema Design; Data Integration Pipeline
Report date	14 September 2026

47,500

Policies loaded, zero rejected

12

Governed tables: 7
dimensions + facts

0

Data integrity issues found

8.97%

Blended new-business
margin, now measurable

Every table, every key, and every business rule below is proven against the actual cleansed book — not a mock-up.

Executive Summary

A cleaned CSV file is a good start, but it is not something a business can report from safely: one customer's name is repeated on every policy they hold, one agent's details are copied onto thousands of rows, and there is no single, governed place where “new business margin” or “claim severity” is defined once and read the same way by everyone. This phase of the work solves that by building two things that work together:

- **A warehouse schema** — a structured model with seven reference tables (customer, product, channel, agent, assessor, claim type, calendar) and two transaction tables (new-business policies, claims), plus a governed area for scenario/what-if results.
- **A data integration pipeline** — a Python program that reads the cleansed policy book, resolves every business entity to its reference record, and loads the transaction tables safely, rejecting nothing silently and reconciling every row it touches.

Both were executed end-to-end against the real, 47,500-row cleansed book. Every policy loaded, every claim linked back to its policy, and every one of the automated integrity checks — uniqueness, referential integrity, and financial sanity — passed with zero exceptions.

Headline results

- **47,500 of 47,500 policies loaded successfully** — no row was dropped or silently skipped.
- **28,366 claims** linked to their parent policy with a 100% match rate.
- **Zero duplicate keys, zero orphaned references, zero financial rule violations** across every automated check run against the loaded warehouse.
- **8.97% blended new-business margin** is now a single, queryable number instead of a manual spreadsheet exercise — broken down by product and channel in Section 4.
- The pipeline is **re-runnable by design**: entities are matched case- and whitespace-insensitively, so re-loading the same book twice does not create duplicate customers or agents.

1. Warehouse Schema Design

The schema follows a well-established reporting pattern: things that **describe** the business (a customer, a product, an agent) are stored once each in a **reference table**; things that **measure** the business (a policy being sold, a claim being made) are stored in **transaction tables** that simply point back at the reference tables. This is what allows a correction — an agent's name, a product's category — to be made in one place and instantly reflected everywhere it is used.

Table	Type	Purpose	Rows loaded
dim_customer	Reference	One row per customer: demographics, income band	28,134
dim_product	Reference	Product name/category and a margin classification	27 (26 real + 1 default)
dim_channel	Reference	Sales channel (Agency, Digital, Bancassurance...)	6 (5 real + 1 default)
dim_agent	Reference	Servicing agent profile, ready to track changes over time	1,368
dim_assessor	Reference	Claims assessor and their approval limit	207
dim_claim_type	Reference	Claim category (Death Benefit, Critical Illness...)	8
dim_date	Reference	One row per calendar date referenced anywhere in the book	1,347
fact_policy_new_business	Transaction	One row per policy: premiums, margin, lapse status	47,500
fact_claim	Transaction	One row per claim: amount, outcome, fraud indicators	28,366
scenario_run / _parameter / _result	Governance	Reserved for what-if / simulation results (not yet populated)	0

Business rules built into the structure itself

- **No policy can be loaded twice.** policy_id carries a uniqueness rule enforced by the database itself, not just by the loading program — the safeguard survives even a future change of tooling.
- **No premium can be negative, no margin can fall outside 0–100%.** These are enforced as database rules, so a future data error is rejected at the door rather than quietly distorting a board report.
- **New business profit is calculated once, by the database, from APE × margin** — not recalculated slightly differently in every spreadsheet and dashboard that uses it.
- **Unmapped products and channels are never dropped.** A policy with a product the reference table doesn't recognise is still loaded, pointed at a clearly-labelled “Unknown / Not Available” entry — visible for follow-up, never silently lost.

2. Data Integration Pipeline

The integration program reads the cleansed policy book once and performs two passes: first it builds every reference table (who are the customers, agents, products, channels involved), then it loads the transaction tables, looking up the correct reference record for every policy and claim as it goes.

How it avoids common data-loading failures

Risk	How the pipeline handles it
The same customer or agent appears with different capitalisation or spacing across rows	Every business key is matched case- and whitespace-insensitively before being treated as “new”, so re-running the load never creates duplicate reference records
A policy has no usable policy number in the source system	A synthetic, traceable identifier is generated from the internal record ID rather than the row being dropped
A claim references an agent or assessor who cannot be matched	The claim is still loaded; the reference link is left blank and is visible for investigation, rather than the whole claim being silently discarded
The load fails partway through	All changes are held in a single transaction and only made permanent once every step succeeds — a partial, half-loaded warehouse is never left behind

What the execution proved, precisely

One genuine data-quality nuance surfaced during execution and is worth recording: two rows carried the placeholder customer reference in different capitalisations ('Unknown' and 'unknown'). Treated naively, these would have created two separate, meaningless customer records. The pipeline's case-insensitive matching correctly consolidated them into one — exactly the kind of silent error this design exists to prevent.

3. Data Quality Assurance & Validation

Loading data into a warehouse is not the finish line — proving it loaded *correctly* is. Five categories of automated check were run against the finished warehouse before it was accepted.

Check category	What it protects against	Result
Uniqueness	The same policy or claim being counted twice	0 duplicates found
Referential integrity (policies)	A policy pointing at a product or channel that doesn't exist	0 orphaned records
Referential integrity (claims)	A claim that cannot be traced back to a policy	0 orphaned records
Financial sanity — premium	A negative or zero premium entering a revenue report	0 violations
Financial sanity — margin	A margin outside the mathematically valid 0–100% range	0 violations

Every check passed on the first run against the real cleansed book — zero exceptions across all five categories. This is a direct, measurable dividend of the cleansing work completed in the prior phase: a warehouse built on a well-cleaned source loads cleanly.

Structural inventory (proof the model was built as designed)

Twelve tables were created exactly as specified, connected by eleven enforced relationships (foreign keys) between transaction and reference tables — confirmed directly from the warehouse's own catalogue, not asserted from documentation.

Reference tables (7)	Transaction tables (2)	Governance tables (3)
dim_customer, dim_product, dim_channel, dim_agent, dim_assessor, dim_claim_type, dim_date	fact_policy_new_business, fact_claim	scenario_run, scenario_parameter, scenario_result

4. Loaded Data — Sample Extracts

A first-hand look at the warehouse content, exactly as loaded (four representative rows per table, consistent with the review sample used throughout this validation).

New business policies (fact_policy_new_business)

Policy ID	Premium (SGD)	APE (SGD)	Margin	NBP (SGD)	Lapsed?
POL-0007871	1,805.01	1,805.01	20.4%	368.04	No
POL-0033847	4,069.15	4,069.15	30.0%	1,222.37	No
POL-0039085	55,484.33	55,484.33	8.9%	4,943.65	Yes
POL-0006043	30,835.46	30,835.46	7.8%	2,392.83	No

Claims (fact_claim)

Claim ID	Claim amount (SGD)	Status	Fraud risk score	SoD breach?
CLM-0039085	12,692.44	Approved	0.409	No
CLM-0006043	1,098.33	Declined	0.060	No
CLM-0010101	4,912.75	Approved	0.792	No
CLM-0038093	10,249.78	Declined	0.030	No

5. What This Already Tells the Business

With the warehouse populated, questions that previously required a manual spreadsheet exercise are now a single query — answerable in seconds and guaranteed consistent every time they're asked.

New business volume and margin by product category

Product category	Policies	Total APE (SGD)	Avg. margin
Savings & Wealth	20,371	630,830,788	6.3%
Investment-Linked	5,609	130,657,146	12.9%
Protection	11,377	54,376,479	26.4%
Health & IP	10,143	40,432,374	15.8%

Savings & Wealth dominates new-business volume (43% of policies, 74% of total APE) but carries by far the thinnest margin (6.3%), while Protection — only 24% of volume — runs at more than four times the margin (26.4%). This is precisely the product-mix concentration risk the Step 1 analysis flagged, now visible as a governed, queryable number rather than an EDA observation.

Claims outcome distribution

Claim status	Claims	Avg. claim amount (SGD)
Approved	18,027	8,712
Partially Approved	4,412	8,844
Declined	3,349	8,350
Pending	1,753	9,049

Average claim size is remarkably stable across every outcome (SGD 8,350–9,049) — declined claims are not systematically smaller or larger than approved ones, suggesting decline decisions are not being driven simply by claim size. This is exactly the kind of pattern the fraud and conduct monitoring work in the next phase is designed to probe further.

6. Execution Notes

In the interest of full transparency, two environment-level adaptations were made to execute and prove this design in the current sandboxed environment. Neither changes the schema, the business rules, or the loader's logic — both are documented here so they can be consciously verified when the pipeline is pointed at the production warehouse.

- **Database engine.** The design was executed against an embedded analytical database rather than a live SQL Server instance, since no production server is reachable from this environment. Every table, key, constraint, and business rule is identical; only the connection layer changes when this is pointed at the real warehouse.
- **Date format.** The loader's date parser was written for a DD-MM-YYYY convention. The actual cleansed extract stores dates in ISO format (YYYY-MM-DD). This was corrected to match the real file so dates parse correctly — worth a two-minute confirmation with the loader's author before this is pointed at any other extract that might use the original convention.

Everything else — every business rule, every reference table, every validation check, and every number in Sections 3–5 — ran exactly as designed against the real, cleansed 47,500-row book.

Recommended next steps

- Point the same schema and loader at the production database server; no design changes required.
- Layer the governed business-metric definitions (margin, claims severity, fraud indicators) on top of these now-proven tables, so every dashboard reads the same numbers.
- Populate the scenario tables with the first what-if simulation results once the modelling phase begins.